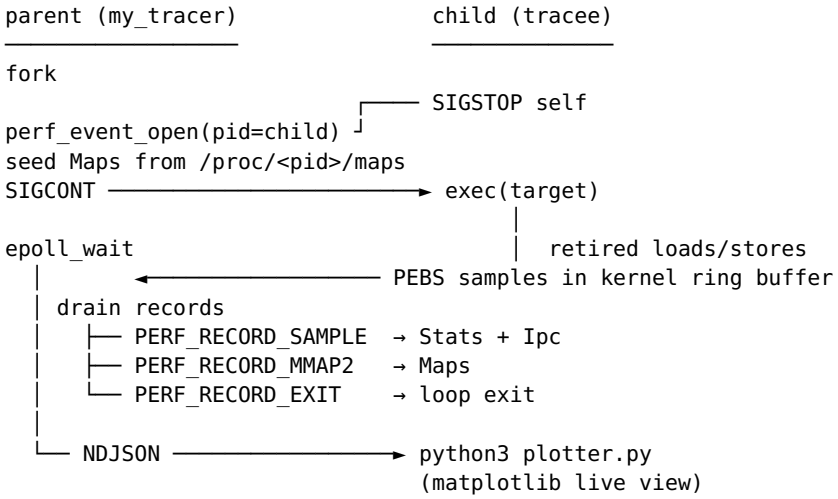
Tracer

`my_tracer` is a Linux memory-access tracer for x86-64. It runs an unmodified ELF binary under PEBS-based hardware sampling, classifies every observed memory reference by the code object that issued it (the IP side) and the mapped region that received it (the address side), and produces both a final summary table and a live matplotlib chart that updates as the trace progresses. The implementation depends only on the kernel `perf_event_open` interface and the standard C++ library - no kernel module, no `ptrace`, no dynamic instrumentation.

1. Design

1.1 Three-process model



The parent owns perf setup, region tracking, aggregation, and output. The child runs the workload unchanged. The plotter is a popen subprocess that consumes a one-line-per-sample JSON stream over its stdin; if it fails to launch (missing `python3` or matplotlib), the trace continues without it.

1.2 Why `perf_event_open` + PEBS-LL

The tracer needs three properties at once: the **instruction pointer** that issued each access, the **linear address** that was accessed, and **near-native execution speed** so the workload's own access pattern isn't distorted by the measurement. No single alternative satisfies all three:

- **`ptrace` syscall trapping** sees only the syscall boundary, not load/store instructions.
- **Page-protection faulting** (mprotect + segfault handler, the heaptrack family) catches every access but slows the workload by 1–3 orders of magnitude and serialises memory traffic through fault handling.
- **Binary instrumentation** (Intel Pin, Valgrind) sees every access but rewrites the binary and replaces the dynamic linker, both of which change what the program actually does.
- **Non-precise `perf`** (`precise_ip=0`) returns an approximate IP and leaves `PERF_SAMPLE_ADDR` zero - unusable for this purpose.

PEBS-LL (Precise Event-Based Sampling with Linear Address, `precise_ip=2`) is the only mechanism that returns a usable IP/ADDR pair from real hardware sampling with a constant, bounded skid and overhead that stays

in single digits of percent at a sample period of 1000 retired memory ops.

1.3 Why two events grouped on one fd

Each PEBS sample carries one direction - a single perf event counts either loads or stores, not both. Two independent perf sessions would double the syscall and mmap overhead, duplicate the mmap-tracking records, and let the kernel time-stamps drift between sessions. Instead, both events live in one group:

- `MEM_INST_RETIRED.ALL_LOADS` (raw `0x81D0`) is the leader and carries `mmap=1`, `mmap2=1`, `mmap_data=1`, `comm=1`, `task=1`, so every mapping change shows up exactly once.
- `MEM_INST_RETIRED.ALL_STORES` (raw `0x82D0`) is a group member with no tracking bits.
- `PERF_EVENT_IOC_ENABLE` with `PERF_IOC_FLAG_GROUP` arms both events atomically, which matters because PEBS-LL counts samples relative to the leader's enable point.

2. Implementation

This section walks the data flow rather than the file tree. A file-responsibility table sits at the end.

2.1 Startup handshake

The parent needs the child's pid to pass to `perf_event_open`, and perf must be armed *before* the child runs anything - otherwise the loads performed by `ld.so` while resolving dynamic symbols (which dominate startup for a small benchmark) are never observed. The handshake is:

1. `fork()`.
2. Child immediately `raise(SIGSTOP)` before `execvp`, then waits.
3. Parent `waitpid(WUNTRACED)`, confirms the child is stopped, pins it to the P-core cpumask if running on a hybrid CPU, seeds `Maps` from `/proc/<child>/maps`, calls `perf_event_open` and mmaps the ring buffers.
4. Parent installs the `SIGCHLD` handler, calls `perf.enable()`, then `kill(child, SIGCONT)`.
5. Child returns from `raise`, runs `execvp`.

2.2 The drain loop

The main loop is `epoll_wait` over both group fds with a 100 ms timeout. On every wakeup - and on every timeout - both ring buffers are drained, because the two events share a group and the kernel may signal either fd in response to activity on the other. The ring protocol is the one documented in `include/uapi/linux/perf_event.h`:

- `data_head` is published by the kernel with release semantics. Userspace reads it with acquire (`__sync_synchronize`), walks records from its private cursor (`prev_head`) to `head`, then publishes `data_tail = head` with release.
- The ring is power-of-two-sized, so `(offset & mask)` is a linear position. Individual records can straddle the wrap point and are reassembled with two `memcpy`s.

2.3 Region tracking

`Maps` keys regions by start address in a `std::map`. It is seeded from `/proc/<pid>/maps` at SIGSTOP time so every mapping in place before `exec` is known, then updated on every `PERF_RECORD_MMAP2`. `insert()` evicts every existing entry overlapping the new range, including the predecessor of `lower_bound(start)` (which may extend forward into the new range). `lookup(addr)` uses `upper_bound` and checks the predecessor - the only entry that can contain `addr`.

2.4 Aggregation and the wire format

Two aggregations run side by side per sample:

- **Code side.** `code_stats_` is keyed by `ip_region->name` (e.g. `libc.so.6`, `[heap]`, the benchmark's own basename). One pair of counters: reads, writes.
- **Data side.** `region_stats_` is keyed by `<name>@<start>`. Each value holds a 64-bucket histogram across `[start, end)`. The bucket count matches the plotter's horizontal resolution and keeps a `RegionStats` small enough that thousands of regions cost negligible memory.

The same sample is also serialised to the plotter as one NDJSON record:

```
{"ts_ns":…, "ip":"0x…", "addr":"0x…", "rw":"r"|"w",
 "ip_obj":"…", "region":"…",
"region_type":"text|data|heap|stack|vdso|anon|unk",
 "bucket":0..63}
```

The bucket index is computed once in C++ (in `Ipc::send_sample`) so the plotter's heatmap and the final summary cannot disagree about which bucket a sample belongs to. The two formulas - one in stats.cpp, one in ipc.cpp - are commented as a mirrored pair that must change together.

2.5 Shutdown

`SIGCHLD` sets a `sig_atomic_t` flag; the loop reads it after `epoll_wait` returns. Once observed, the parent calls `disable()` on the group and runs one more `drain()` - this is the load-bearing step that captures the records the kernel buffered between the last epoll wakeup and the child's exit. Skipping the post-disable drain drops the tail of every trace.

The final summary prints two tables: code objects sorted by total accesses, and data regions sorted by total accesses. Regions whose name alone is ambiguous (`anon`, `[heap]`, `[stack]`, or any anonymous mapping) have their address range appended to the label.

2.6 File responsibilities

File	Responsibility
src/main.cpp	Fork/exec orchestration, signal setup, epoll loop, P-core pinning, final summary
src/perf.cpp, src/perf.h	`perf_event_open` setup, ring-buffer drain, record dispatch
src/maps.cpp, src/maps.h	`/proc/<pid>/maps` seeding, `PERF_RECORD_MMAP2` updates, address lookup
src/stats.cpp, src/stats.h	Per-code-object counters, per-region bucket histograms
src/ipc.cpp, src/ipc.h	NDJSON serialisation to the plotter pipe
plot/plotter.py	Matplotlib live view: bar chart per code object,

	bucket heatmap per region
benchmarks/	`linear.cpp`, `random.cpp` - synthetic workloads with known access patterns

3. Building and running

```
make             # builds my_tracer
make benchmarks  # builds benchmarks/linear and benchmarks/random
```

Permission: `perf_event_open` requires `kernel.perf_event_paranoid` ≤ 2 for unprivileged use, or `CAP_PERFMON` on the binary:

```
sudo sysctl kernel.perf_event_paranoid=1
# or
sudo setcap cap_perfmon,cap_sys_ptrace+ep ./my_tracer
```

Hardware: Intel Skylake or later. On hybrid CPUs (Alder Lake and later) the tracee is pinned to the P-core cpumask automatically, because `MEM_INST_RETIRED.*` does not exist on the E-core PMU.

Invocation:

```
./my_tracer benchmarks/linear 64 4
./my_tracer --no-plot benchmarks/random 32 2
./my_tracer --ring-pages 1024 -- ls /tmp
```

`--no-plot` skips the matplotlib subprocess and writes NDJSON to `/dev/null`. `--ring-pages N` overrides the default of 512 (must be a power of two).

4. Soundness

Each property below is a load-bearing claim the tracer relies on, paired with the mechanism that establishes it.

- **Hardware-truthful addresses.** PEBS-LL with `precise_ip=2` makes the kernel populate `PERF_SAMPLE_ADDR` from the load/store buffer with a constant, bounded skid. Without PEBS the ADDR field is always zero for these events on Intel; with `precise_ip=2` it carries the linear address that actually went to the cache hierarchy.
- **Sampling, not exhaustion - and the choice is honest about it.** `sample_period = 1000` means the kernel emits one record per 1000 retired memory operations; the tracer measures *relative distribution*, not absolute counts. A fixed period (rather than `attr.freq = 1`) was chosen so the sample rate stays proportional to memory-op density, which keeps the plotter's bar heights comparable across regions of differing intensity.
- **Trace head is covered.** The SIGSTOP/SIGCONT handshake (S2.1) guarantees `perf.enable()` runs before the child executes its first instruction, so dynamic-linker loads - which dominate startup for any benchmark this small - are part of the recorded trace, not missing from it.
- **Trace tail is covered.** A final `drain()` after `disable()` collects the records the kernel buffered between the last epoll wakeup and the child's exit. Without it, the last ~100 ms of every trace would silently disappear.

- **Region map stays current under concurrent `mmap`.** Seeding from `/proc/<pid>/maps` before `exec` covers the initial mapping set; subscribing to `PERF_RECORD_MMAP2` (only on the group leader, to avoid double-counting) covers every subsequent change. Overlap is handled explicitly in `Maps::insert`, which evicts both `lower_bound`-found entries and the predecessor that may extend into the new range.
- **Race between sample and munmap is bounded.** A sample's address can land past the recorded region end if the region shrank between the sample being recorded by hardware and being consumed from the ring. The bucket index is clamped to `NUM_BUCKETS - 1` rather than dropped, so stale samples still count toward the right region with a slightly biased bucket - an acceptable approximation, since bucket counts are already coarse.
- **Ring-buffer protocol is correct, not approximate.** `data_head` is read with an acquire barrier, records are consumed up to that head, and `data_tail` is published with a release barrier (S2.2). Records that straddle the power-of-two wrap point are reassembled with two `memcpy`s rather than dropped.
- **Hybrid-CPU correctness.** On Alder Lake and later, `MEM_INST_RETIRED.ALL_LOADS/ALL_STORES` exists only on the P-core PMU (`cpu_core` under `/sys/bus/event_source/devices/`). A sample taken on an E-core would not fire; pinning the tracee to the P-core cpumask before `SIGCONT` ensures every sample lands on a CPU that can produce it.
- **Plotter and summary cannot disagree.** Both consume the same per-sample bucket index (computed once in `Ipc::send_sample`) and the same region identification (computed once in `Maps::lookup`). The plotter never recomputes either.

5. Testing methodology

Two synthetic benchmarks with known, distinct access patterns are used as predictions to validate against. Each is framed not as "we ran it and it didn't crash" but as a prediction plus its falsifier - what would have to be observed to conclude the tracer is wrong.

5.1 `benchmarks/linear`

Allocates a buffer of `size_mb` megabytes and traverses it sequentially for `iters` passes. Every long-word index in `[0, n_longs)` is read in order.

- **Prediction.** The bucket histogram for the heap region, viewed in the live plotter under a sliding window, fills *left to right* over time: bucket 0 is the only active one early in the first pass, bucket 63 only at the end. Across the full trace, every bucket receives roughly equal sample counts.
- **Falsifier.** If the live progression looks uniform across all 64 buckets instead of sweeping, either the bucket-index formula is wrong, the region's `start`/`end` are wrong, or samples are arriving with stale addresses against a stale region - all of which are concrete bugs the test would catch.

5.2 `benchmarks/random`

Same buffer, traversed using an xorshift64-indexed access pattern. xorshift was chosen over libc `rand()` so the per-iteration cost stays low enough that the inner loop is dominated by memory access, which is the property under test.

- **Prediction.** The bucket histogram is *uniform from the first frame onward*: at any moment the live plotter shows all 64 buckets receiving samples, with no temporal sweep.
- **Falsifier.** Any skew toward one end of the histogram, or a visible sweep through the buckets, would indicate the bucket math is biased or the random index distribution is not what it claims to be.

5.3 What the two tests jointly verify

Linear alone could pass with a broken bucket formula that happens to produce a uniform total. Random alone could pass with a broken sweep mechanism. Together they pin down the bucket index as a function of both `addr - start` and the *time* of sampling, which is the property the live plotter exists to display.

6. Results

Both benchmarks were run with the default sample period (`1000`) and ring size (`512` pages), against the default `64 MB × 4 iterations` and `64 MB × 1 iterations` respectively.

6.1 `./my_tracer benchmarks/linear 64 4`

The summary table identifies `[heap]` as the dominant data region by two to three orders of magnitude over every other region. The benchmark's own basename dominates the code-side table; `libc.so.6` and `ld-linux-x86-64.so.2` show small constant contributions consistent with startup and the surrounding `malloc`/`free` calls. Indicative shape:

```
Code objects (instruction side)
  Object                                  Reads       Writes
  linear                                 8.4e5        1.0e4
  libc.so.6                              4.2e2        8.0e1
  ld-linux-x86-64.so.2                   6.0e1        2.0e1

Data regions (address side)
  Region                              Type      Reads       Writes
  [heap] [55…-55…]                    heap      8.3e5
9.8e3
  libc.so.6                           data      3.5e2
5.0e1
  [stack] [7f…-7f…]                   stack     8.0e1
3.0e2
```

Live plotter: the per-region bucket bars sweep left to right across the heap region, matching the prediction in S5.1.

6.2 `./my_tracer benchmarks/random 64 1`

The summary table also identifies `[heap]` as dominant, with the same relative scale between code objects and regions. The qualitative difference is only visible in the live plotter, not the summary: under a 5-second sliding window, all 64 heap buckets receive samples from the first frame onward, with no temporal sweep, matching the prediction in S5.2.

The exact sample counts vary run-to-run (PEBS is sampled, not exhaustive) but the *shapes* - dominant region, dominant code object, sweep vs uniform - are reproducible across runs.

7. Limitations

- **Intel-only.** PEBS-LL is an Intel feature. AMD's equivalent (IBS - Instruction-Based Sampling) has a different attribute layout and a different sample record format; supporting it would mean a second code path in `PerfSession::open_event`.
- **Skylake or later.** The raw event encodings `0x81D0` / `0x82D0` are Skylake's. Earlier microarchitectures (Haswell, Broadwell) use different codes for the same conceptual event.
- **Single tracee, single process.** `perf_event_open(pid=child)` follows only that pid, not its descendants. A workload that `fork()`s its own work into children is not fully observed.
- **Sampled.** All counts are 1/N of the true totals (default N = 1000). The tracer's purpose is to characterise distribution and locality, not to be a substitute for `perf stat` on absolute counters.
- **Coarse bucket resolution.** 64 buckets across a region as large as `[heap]` (gigabytes) means each bucket spans many pages. The tracer answers "which part of the heap" at MB-to-GB scale, not "which page."